Lab 10 — Create an Azure SQL Database

In this lab you deploy a fully managed Azure SQL Database preloaded with the AdventureWorksLT sample data and query it directly from the browser — no database server to install, patch, or back up. This maps to the AZ-900 skill area **Describe Azure architecture and services** — database services and the PaaS shared-responsibility model.

Run on https://portal.azure.com — sign in with your Azure free account (https://azure.microsoft.com/free).

Step 1 — Start the SQL Database wizard

1. Click **Create a resource** > search **SQL Database** > **Create** > **SQL Database**.
2. On the **Basics** tab set:

- Subscription: your free subscription
- Resource group: `rg-az900-labs` (create it if it doesn't exist)
- Database name: `db-az900`

Step 2 — Create a new logical server

1. Under **Server**, click **Create new** and enter:

Field	Value
Server name	`sql-az900-<yourinitials><random>` (e.g. `sql-az900-tan4821` — globally unique)
Location	`Southeast Asia`
Authentication method	`Use SQL authentication`
Server admin login	`sqladmin`
Password	a strong password — write it down, you need it in Step 6

1. Click **OK**.

The logical server is a management boundary (logins, firewall rules) — it is not a VM you can log into.

Step 3 — Choose a cheap compute + storage tier

1. Back on **Basics**, set **Workload environment** = `Development` if offered.
2. Under **Compute + storage**, click **Configure database**.

3. Choose one of the two lab-friendly options:

- **Service tier** = `Basic` (5 DTUs, 2 GB) — simplest and about USD 5/month, or
- **Service tier** = `General Purpose (Serverless)` with **Auto-pause delay** = 1 hour and min/max vCores at the lowest values — bills per second of use and pauses when idle.

1. Click **Apply**. On **Basics**, set **Backup storage redundancy** = `Locally-redundant backup storage`.

Serverless with auto-pause is the cloud consumption model in miniature: pay only while the database is active.

Step 4 — Configure networking

1. Click **Next: Networking** and set:

- Connectivity method: `Public endpoint`
- Allow Azure services and resources to access this server: `Yes`
- Add current client IP address: `Yes`

1. Leave **Connection policy** at Default.

By default a SQL server's firewall blocks everything; these two rules admit Azure services (needed by Query editor) and your own machine.

Step 5 — Load sample data and create

1. Click **Next: Security** — leave defaults (make sure **Microsoft Defender for SQL** = `Not now` to avoid charges).
2. Click **Next: Additional settings** and set:

- Use existing data: `Sample` (this loads the **AdventureWorksLT** database; accept the collation prompt)

1. Click **Review + create** > **Create**. Deployment takes 2–5 minutes; then click **Go to resource**.

Step 6 — Query the database from the browser

1. On the **db-az900** database blade, click **Query editor (preview)** in the left menu.
2. Sign in with **SQL server authentication**: login `sqladmin` and your password from Step 2.
3. In the query pane, run:

```
SELECT TOP 10 Name, ListPrice FROM SalesLT.Product ORDER BY ListPrice DESC;
```

1. Click **Run** — the 10 most expensive AdventureWorks products appear in the Results grid.

You just queried a production-grade SQL database entirely from a browser. If you chose serverless and the DB auto-paused, the first query may take ~30 seconds to resume — that's the trade-off for paying nothing while paused.

Step 7 — Observe what PaaS removed from your plate

1. On the database's **Overview** blade note: compute tier, storage used, and the server name `sql-az900-....database.windows.net`.
2. Look at the left menu: there is no "Restart VM", no OS, no patching schedule. Under **Data management > Backups** (on the server) note that automated backups with point-in-time restore are already running — you configured nothing.

With SQL on an Azure VM (IaaS) you would patch Windows and SQL Server yourself; with Azure SQL Database (PaaS), Microsoft handles the OS, engine patching, HA, and backups.

Step 8 — Check the server firewall

1. From the database **Overview**, click the **Server name** link to open the logical server.
2. In the left menu click **Security > Networking**.
3. Under **Firewall rules**, see the rule with your client IP (added in Step 4) and the checkbox **Allow Azure services and resources to access this server**.
4. Note you could delete the client rule to instantly cut off your own access — the firewall is the outer gate before SQL authentication even happens.

Defense in depth: network firewall first, then authentication, then database permissions.

Clean up

Delete the database and the logical server (keep resource group `rg-az900-labs` — it is reused until Lab 16). In Cloud Shell:

```
az sql db delete --resource-group rg-az900-labs --server
sql-az900-<yourinitials><random> --name db-az900 --yes
az sql server delete --resource-group rg-az900-labs --name
sql-az900-<yourinitials><random> --yes
```

Or in the portal: open `rg-az900-labs`, tick **db-az900** and the SQL server, click **Delete**. Basic tier bills daily, so don't leave it running.

What you learned

- **Azure SQL Database** is a PaaS relational database: Microsoft manages the OS, patching, high availability, and automated backups.

- A **logical SQL server** is an administrative boundary for logins and firewall rules, not a machine.
- **DTU (Basic)** vs **vCore serverless with auto-pause** are two purchasing models — serverless embodies pay-per-use consumption pricing.
- The **server firewall** blocks all access by default; you explicitly allow client IPs and Azure services.
- **Query editor** in the portal lets you run T-SQL with zero client tooling — useful for quick verification and demos.

Files in this lab folder

File	What it is
`lab10-queries.sql`	All the Query editor statements for this lab, ready to paste.

The scripts are optional conveniences — the lab steps above teach the same thing command by command. Upload a script to Cloud Shell (**Manage files > Upload**), or clone this repo there, then run it with `bash <script-name>`.